

CSE 153

Final Project

Symbolic Conditional & Unconditional Music Generation

Jay Gao, Billy Dang, Ryan Park, Muddassir Ahmadzada

Symbolic Unconditional Generation



What is it?

- Symbolic Unconditional Gen. is a method utilized to generate new music sequences (MIDI) by learning and training on patterns of note sequences given from a music dataset.

How it works

- Training: A model is trained to learn the probability distribution of a musical set, where the distribution varies across the sequence of notes, durations, and chords.
- Sampling & Generation: A seed token (random note) is used to start the music generation. The model then predicts the next note / token repeatedly, sampling from our probability distribution. This now creates a new music file that closely resembles the training data but is newly composed.

Part 1. Our Dataset



Context

For both of our models, we used the **MAESTRO** (MIDI and Audio Edited for Synchronous Tracks and Organizations) dataset along with Dr. McAuley's provided tracks from Module 3. The **MAESTRO** dataset consists of approximately **200 hours** of classical piano performances from the **International Piano-e-Competition** (origin of Prof. McAuley's unknown).

MAESTRO Details

- Performances were captured using Yamaha Disklaviers, which are high-end acoustic grand pianos with built-in high-resolution MIDI recording capabilities
- The dataset contains both audio and MIDI recordings, with fine alignments (approximately 3ms accuracy) and detailed annotations including note velocities and pedal positions.
- The dataset covers music from 17th to early 20th century composers, all professionally performed and curated to avoid duplicate compositions across train, val, and test splits.
- As the data set is split into multiple folders of MIDI files by year, we decided to just use those found in the "2004" folder

Part 1. Our Dataset



Discussion & Code [EDA]

We begin by conducting exploratory data analysis on our MIDI folders and files as for any dataset, checking completeness and general trends / details.

Code to extract MIDI features (pitch, etc.):

```
def extract_midi_features(path):
    pm = pretty_midi.PrettyMIDI(path)

    all_pitches = []
    all_velocities = []
    all_durations = []

    for inst in pm.instruments:
        for note in inst.notes:
            all_pitches.append(note.pitch)
            all_velocities.append(note.velocity)
            duration = note.end - note.start
            all_durations.append(duration)

    if len(all_pitches) == 0:
        pitch_min = pitch_max = pitch_mean = np.nan
        vel_mean = np.nan
        dur_mean = np.nan
        total_notes = 0
    else:
        pitch_min = np.min(all_pitches)
        pitch_max = np.max(all_pitches)
        pitch_mean = np.mean(all_pitches)
        vel_mean = np.mean(all_velocities)
        dur_mean = np.mean(all_durations)
        total_notes = len(all_pitches)

    return {
        'filename': os.path.basename(path),
        'total_notes': total_notes,
        'pitch_min': pitch_min,
        'pitch_max': pitch_max,
        'pitch_mean': pitch_mean,
        'vel_mean': vel_mean,
        'dur_mean': dur_mean
    }
```

Check for empty midi files:

```
zero_mc = df_mcauley[df_mcauley["total_notes"] == 0]
print("McAuley - files with zero notes:")
print(zero_mc["filename"].tolist(), "\n")
zero_ma = df_maestro[df_maestro["total_notes"] == 0]
print("Maestro - files with zero notes:")
print(zero_ma["filename"].tolist())

✓ 0.0s

McAuley files with zero notes:
[]

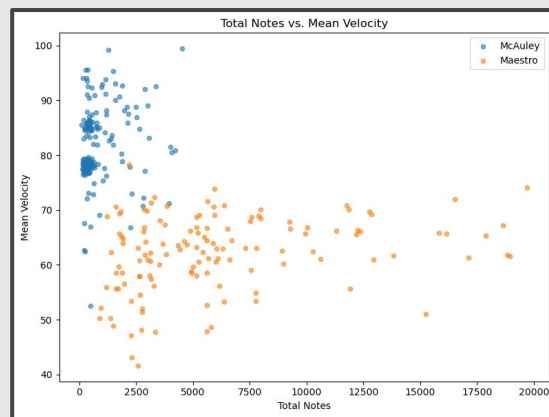
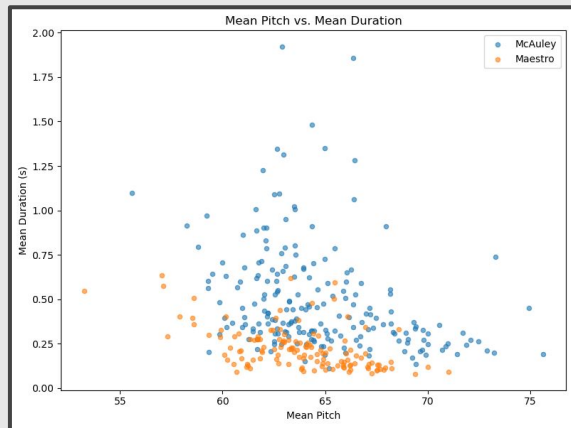
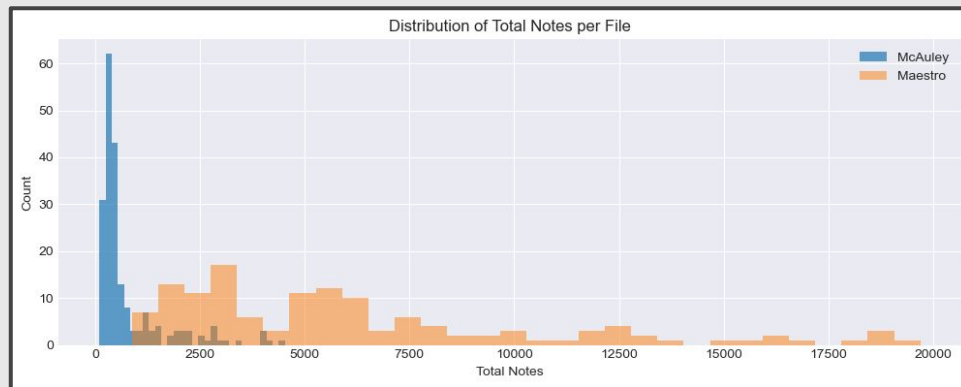
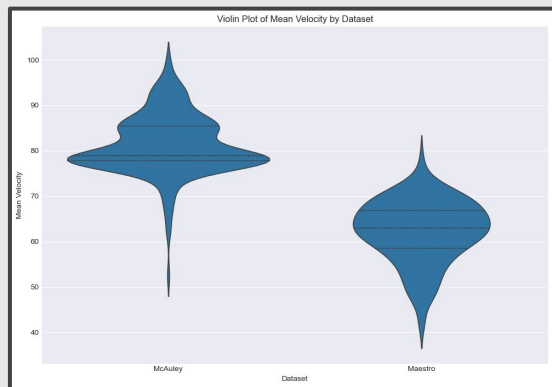
Maestro files with zero notes:
[]
```

Examine general summary statistics of both datasets:

	50%		max		mean		min		std	
dataset	Maestro	McAuley	Maestro	McAuley	Maestro	McAuley	Maestro	McAuley	Maestro	McAuley
feature										
dur_mean	0.200629	0.394827	0.635987	1.920013	0.225208	0.497030	0.080326	0.111073	0.119467	0.300522
pitch_max	98.500000	85.000000	108.000000	116.000000	97.015152	87.463054	77.000000	74.000000	6.794682	9.078446
pitch_mean	63.853115	64.013407	71.026357	75.618120	63.746429	64.687118	53.238453	55.594012	2.908078	3.419373
pitch_min	27.000000	41.000000	38.000000	60.000000	27.196970	38.891626	21.000000	21.000000	3.975957	8.235283
total_notes	5182.500000	414.000000	19689.000000	4529.000000	6174.484848	769.793103	875.000000	103.000000	4502.784966	866.506698
vel_mean	62.987031	78.858238	78.207384	99.409583	62.236993	81.260089	41.650738	52.474333	6.863812	6.558564

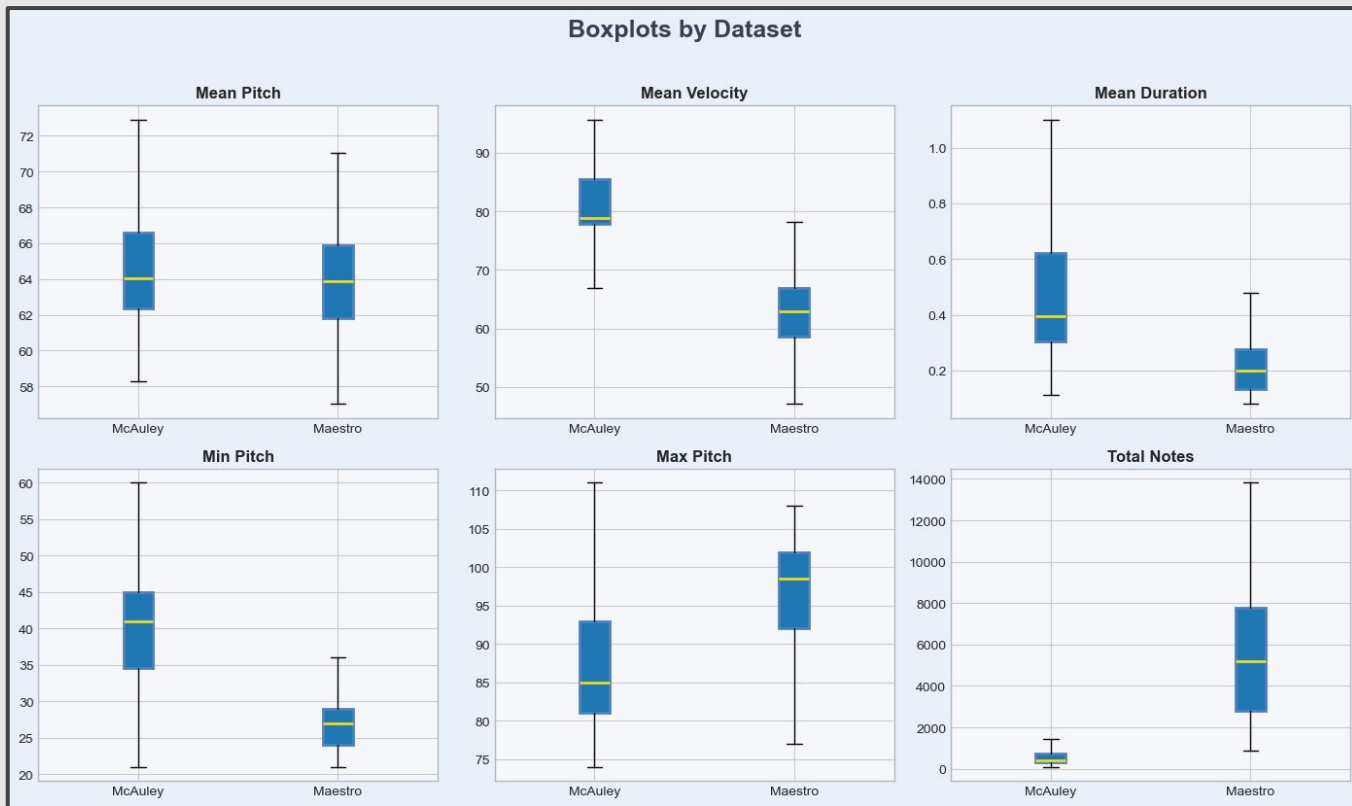
Part 1. Our Dataset

Discussion & Code [EDA Cont.]



Part 1. Our Dataset

Discussion & Code [EDA Cont.]



Part 1. Our Dataset



Discussion & Code [Processing & Preparing for Usage]

Combine & randomly split data for train/test (90/10):

```
# 1. Get all MIDI files

# Get files from the Maestro & Mcauley dataset
path_mcauley = glob.glob("../mcauley_data/*.mid")
path_maestro = glob.glob("../maestro/2004/*.midi")
print(f"Found {len(path_maestro)} MIDI files in Maestro dataset and {len(path_mcauley)} in Mcauley dataset.")

#Combine the lists of MIDI files
all_midi_files = path_mcauley + path_maestro

# 2. Shuffle and Split the files
random.seed(3)
random.shuffle(all_midi_files)

#Using sklearn's train_test_split to create train/test splits (90/10 split)

train_files, test_files = train_test_split(all_midi_files, test_size=0.1, random_state=3)
```

```
# 3. Create the Dataset and DataLoader w/ miditok library REMI tokenizer

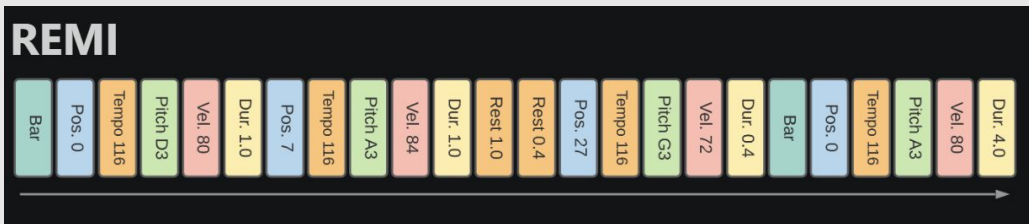
tokenizer = REMI()

train_dataset = DatasetMIDI(
    files_paths=train_files,
    tokenizer=tokenizer,
    max_seq_len=1200,
    bos_token_id=tokenizer["BOS_None"],
    eos_token_id=tokenizer["EOS_None"],
)

test_dataset = DatasetMIDI(
    files_paths=test_files,
    tokenizer=tokenizer,
    max_seq_len=1200,
    bos_token_id=tokenizer["BOS_None"],
    eos_token_id=tokenizer["EOS_None"],
)

collator = DataCollator(tokenizer.pad_token_id)
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True, collate_fn=collator)
test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False, collate_fn=collator)
```

Employ REMI tokenizer from MidiTok library



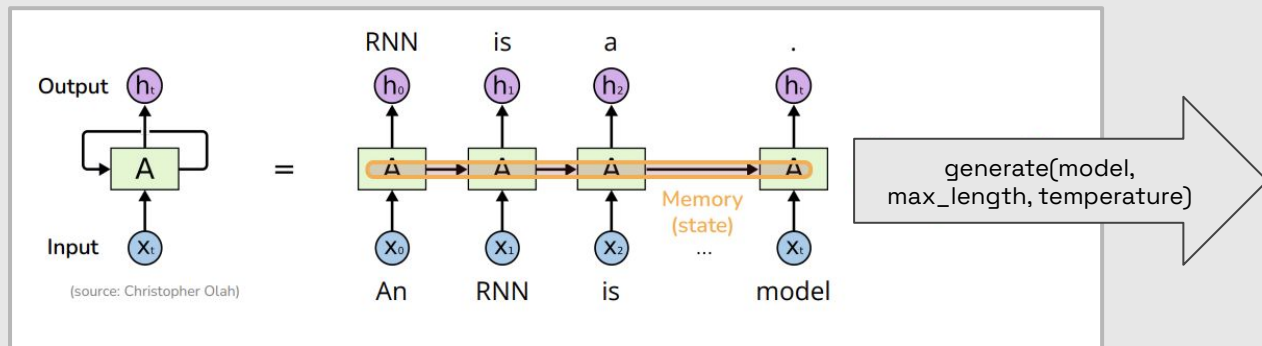
- **What?** “REMI represents notes as successions of *Pitch*, *Velocity* and *Duration* tokens, and time with *Bar* and *Position* tokens.” (Huang & Yang, 2020)
- **Why?** Has been shown to outperform “Midi-like” tokenization models in generating piano music and aims to be more musically “intuitive” using a more structured sequence.
- **Limitations:** REMI uses many tokens to represent a musical note which leads to longer musical pieces consisting of lots of tokens → As models have a finite context window, they may struggle to capture musical relationships spanning across a longer sequence (which we will see later down the line)

Part 2. Modeling

Context [Inputs, Outputs, Goals, & Model Discussion]

As language models provide a reliable framework for Machine Learning problems whose outputs can be represented as a sequence of discrete tokens (like the REMI representation we will be working with) we'll leverage that for our purpose of symbolic music generation task.

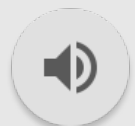
Example framework of a RNN that takes in discrete tokens from REMI as **input** (and uses PyTorch's DataLoader to batch and feed the input)



The model is trained using a standard supervised learning approach to **output** a sequence of tokens (which will be converted back to Midi & .wav files). In the training step, the model is tasked w/ predicting the next token in the sequence at each step w/ a **cross-entropy loss function** to measure the difference between predicted and actual tokens. We also use the **Adam optimizer** to adjust weights during backpropagation.



Some generated token sequence i.e. [1, 143, 150, 35, ...]



Raw implementation of RNN w/ limited batching, embedding dim, etc. (Not great)

Part 2. Modeling



Discussion & Code [Optimization & Parameter Tuning]

As we deployed the following neural networks, there were many parameters we needed to optimize to find a good balance between model performance, training time, and computational resources.

1. **embedding_dim:** Defines the size of the dense vector used to represent each REMI token where larger values allow the model to learn richer & more “nuanced” relationships”, we initially considered 64 before scaling up by multiples of 2.
2. **hidden_dim:** Specifies the # of features in the hidden state of each cell & acts as the “memory capacity” where a larger value gives the model more space to store information from past sequences. Same considerations and scaling as embedding_dim
3. **num_layers:** Represents the number of NN layers stacked on top of each each, allowing model to learn hierarchical representations. More layers meant a deeper network w/ more parameters for learning.
4. **dropout:** Dropout rate sets a fraction (i.e. 0.1) of the layer’s input units to 0 at each update as a form of regularizing the data to prevent overfitting
5. **lr (learning rate):** Controls how much the model’s weight are adjusted w/ respect to the loss gradient
6. **num_epochs:** Number of passes through the training data, monitored using the training & validation loss
7. **batch_size:** Number of training samples used in each forward/backward pass
8. **max_seq_len:** Defined the maximum number of tokens that a sequence fed to the model will contain during training (with longer sequences truncated and shorted ones padded. Allows the model to potentially learn relationships between more distance musical events

Part 2. Modeling

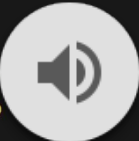


Discussion & Code [RNN vs GRU vs LSTM]

We tested 3 different neural network implementations, each with their own advantages and disadvantages to output the following results:

```
class MusicRNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_layers):
        super(MusicRNN, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.rnn = nn.RNN(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=False,
            nonlinearity="tanh" # Unidirectional RNN
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (batch_size, seq_length)
        x = self.embedding(x) # (batch_size, seq_length, embedding_dim)
```



The most simple neural network architecture of the three, our group saw slightly faster computes during training - though max_seq_length, layers, and dimensionality were still the most significant contributors to computational cost. The RNN performed rather poorly, likely due to its limited ability to “remember” information from steps that occurred long ago.

```
class MusicGRU(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_layers):
        super(MusicGRU, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.rnn = nn.GRU(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=False,
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)
```



We also tried to use a Gated Recurrent Unit architecture to handle long-term dependencies while not being as computationally intensive as LSTMs. However, its fewer parameters (due to fewer gates) did not seem to “learn” music well as evidenced by the output file above. We stopped further fine-tuning of this model early as it’s attempt at being an “in-between” solution of an RNN vs LSTM proved to be rather inefficient.

```
class MusicLSTM(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_layers):
        super(MusicLSTM, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.rnn = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            dropout=0.1,
            bidirectional=False, # Unidirectional RNN
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (batch_size, seq_length)
        x = self.embedding(x) # (batch_size, seq_length, embedding_dim)
```



Finally, we have our Long Short-Term memory architecture which sought to tackle the complexity issue with it’s cell structure that includes a **Forget Gate**, **Input Gate**, and **Output Gate**. Due to the amount of parameters within this model, it was rather challenging to train and consistently tune the model due to the time & computational demand. However, it would prove to be effective at capturing long term dependencies with just two layers.

Part 2. Modeling



Discussion & Code [Computational Limitations]

As mentioned earlier in our sections, parameters and architecture were instrumental in model improvements, however at the cost of time and computational complexity.

- On cpu-based computation, we saw training times above 30 minutes (on just a subset of our current dataset) and a rather low dimensional model (with lower hidden/dim. embeddings, layers, etc.) As we hoped to increase dimensionality and training data, leveraging CUDA cores was the only way to overcome this.
- Even on CUDA cores, the larger training dataset and increased dimensionality saw training/testing times hover around the 5-10 minute mark. This made trying new models, tuning their respective parameters and data extremely difficult due to time and resource constraints. For example, our group attempted to implement an attention-based weight system to our RNN models (Transformer architecture) for better contextual understanding; however, it's $O(L^2)$ complexity (where L = sequence length) required too much memory for our GPUs.

Evaluation



Why our method?

- We have two baselines, Uniform and Unigram.
- Uniform Baseline: We sample each token with equal probability, where the perplexity would equal to vocabulary size.
- Unigram Baseline: Sample tokens i.i.d from their overall training frequencies.
- For our LSTM / RNN / GRU model to be considered “good”, our perplexity was greatly lower than uniform or unigram baselines, meaning it had learned, and applied it.

What makes our music “good”?

- The properties that would classify our output to be “good” ranges from the analysis of note sequences, pitch transitions, rhythm, and low validation loss.
- Our model assigns high probability to note-sequences, whilst it produces melodies that has smooth pitch transitions. Perplexity measures how well our model would predict tokens.
- Check for generated token frequencies to match with data sets and that short melodies would appear.

Evaluation



```
Evaluating on device: cuda

► vocab_size = 282

► Sample batch shape (input_ids): torch.Size([4, 1023])
First sequence (first 10 tokens): [ 1  4 217 43 111 128  4 189 48 112]
Top-5 predicted tokens for first seq at t=0: [148 16 116 108 65] [0.00382762 0.00379679 0.00379187
Actual next token at t=1: 4
► If the LSTM is learning, the actual next token should have probability > ~1/vocab_size.
```

```
Uniform Baseline CE (nats/token): 5.6419
Uniform Baseline Perplexity      : 282.0000
```

```
Counting train tokens for Unigram: 100%|██████████| 49/49 [00:01<00:00, 33.99it/s]
Scoring Unigram on Val: 100%|██████████| 3/3 [00:00<00:00, 49.18it/s]
Unigram Baseline CE (nats/token): 4.2458
Unigram Baseline Perplexity      : 69.8094
```

```
Evaluating trained LSTM on Val: 100%|██████████| 3/3 [00:00<00:00, 12.56it/s]
LSTM Validation CE (nats/token): 3.3518
LSTM Validation Perplexity       : 28.5539
```

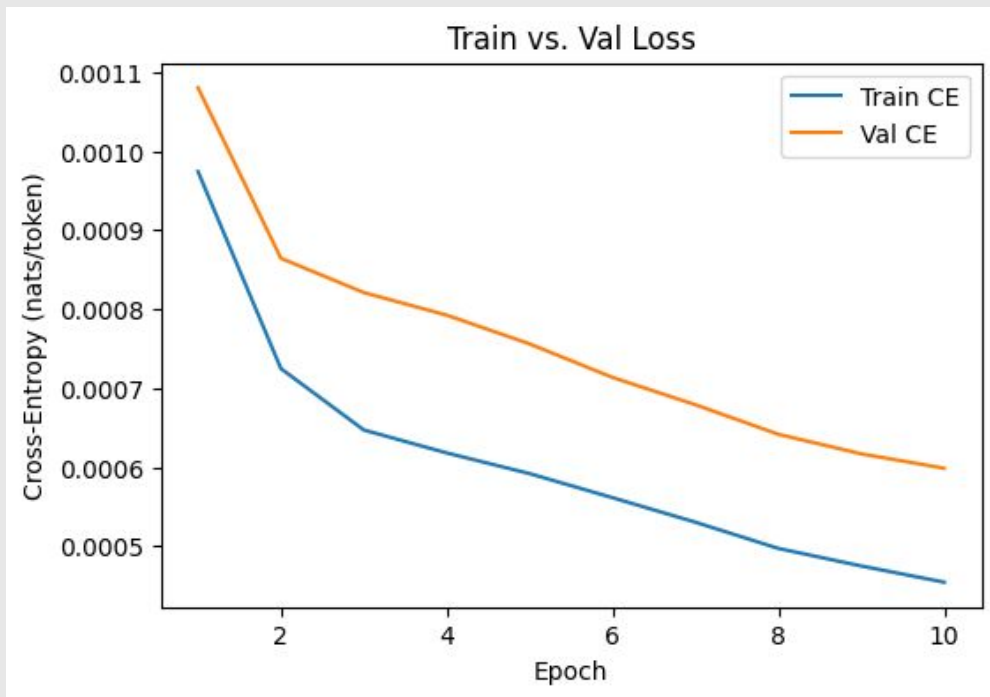
```
KL(real||LSTM_gen) over 1000 tokens: 11.3370
```

Validation Evaluation Table

Model	CrossEntropy (nats/token)	Perplexity
Uniform	5.641907	282.000000
Unigram	4.245768	69.809381
LSTM	3.351795	28.553942

As can be seen from the image, the perplexity from our LSTM model is greatly under both baseline models, indicating that our model had learned and improved on its data.

Evaluation



We plotted a graph to visualize the train loss and value loss per last ten epochs, as we approach the end of the last ten epochs, the model slowly loses less and less meaning it the learning has reached a point where its learning less rather than the first ten epochs where the drop in learning loss was more significant.

Part 4. Discussion of Related Work



- **The Data:** As our dataset was a combination of Prof. McAuley's and a portion of the Maestro data, a direct comparison cannot be made. However, similar research papers on the modeling and generation of piano music pull from various composer performances on the internet as well to train their models. Additionally, these papers saw a far larger file size and complexity (with training sets alone consisting of 700+ performances compared to our ~300)
- **Approaches:** Other research papers that advanced the work and development of music generation saw further data augmentation, tokenization improvements, and deeper models trained on vastly more data to tackle the task. For example, in order to better capture and model the complex nature of music, researchers (Simon & Oore, 2017) used a RNN-based LSTM model that generates expressive timings and dynamics via a stream of midi files. They also applied time stretching and transposition in order to create more training examples and split performances into 30 seconds, finding that the model was still capable of generating longer performances without falling off. → Our group actually reached a similar conclusion when adjusting our data and models as `max_seq_length`, a key part of how we portioned training sequences, was initially helpful to increase. However, extending the sequence beyond ~1200 lead to not only drastically increased training resources, but hindered musical quality.
- **Approaches:** Another paper saw the use of a transformer-based model for symbolic music generation (Ens & Pasquier, 2008) which utilizes the self-attention mechanism to weigh the importance of different parts of the input data. With their ability to essentially “pay attention” these models can better handle longer sequences and are considered the SOTA models.

<https://www.arxiv.org/pdf/2008.06048>

<https://magenta.tensorflow.org/performance-rnn>

<https://arxiv.org/pdf/1809.04281>

Task 2: Symbolic Conditioned Generation



Symbolic, Conditioned Generation



What is it?

Symbolic conditioned generation is a method used to generate new symbolic music sequences (MIDI) based on a given input or prompt. The model is trained to learn musical patterns from tokenized note sequences and generate coherent continuations conditioned on a starting input sequence extracted from an existing MIDI file.

How it works

Training: A LSTM model is trained on a large set of tokenized MIDI files. Each file is then converted into a sequence of musical tokens. The model learns to predict target note sequences (e.g., bass lines) based on input sequences (e.g. melody slices), capturing temporal, and rhythmic relationships

Sampling & Generation: To generate music, we provide the model with a sequence of start tokens extracted from an existing MIDI file. The model then learns to predict a corresponding bass line , conditioned on the given input. This produces a continuation that fits the musical style and structure of the starting sequence, resulting in a new, coherent MIDI composition.

Our Objective: Train an LSTM to take a slice of a short melody (the highest pitch), and predict the corresponding bass line (the lowest pitch)

Part 1. Our Dataset



Context

For both of our models, we used the **MAESTRO** (MIDI and Audio Edited for Synchronous Tracks and Organizations) dataset along with Dr. McAuley's provided tracks from Module 3. This dataset consists of approximately **200 hours** of classical piano performances from the **International Piano-e-Competition**.

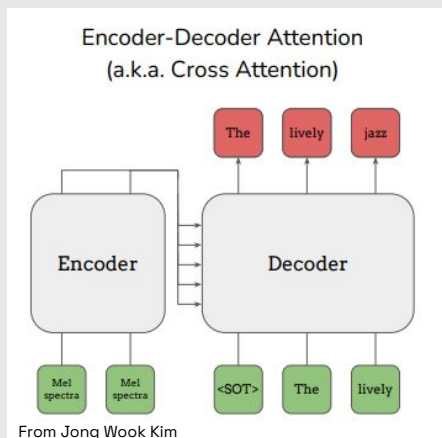
Selection Process:

We experimented with three choices: training on MAESTRO alone, Dr. McAuley's module 3 dataset alone, and a combined set, before settling on our final dataset. Trained only on MAESTRO, the LSTM performed somewhat satisfactorily but could not generalize beyond classical-style passages because MAESTRO's vast Disklavier recordings still lacked modern and jazz hints, while also taking much longer to build. However, McAuley's smaller dataset of precurated piano MIDI files offered higher genre variety but was too small in itself, leading to overfitting and poor model performance. By specifically mixing the 2004 MAESTRO files with Dr. McAuley's files, we combined MAESTRO's depth with the variety of the Module 3 dataset. As we observed, this blend of the Dataset folder "Assignment2DS" always had higher validation accuracy ($\approx 28\%$) than either the MAESTRO-alone ($\approx 18\%$) or McAuley alone ($\approx 12\%$), and indeed proves that it is well balanced, which supported our building/training.

Part 2: Modeling

Context [Inputs, Outputs, Goals, & Model Discussion]

For symbolic condition music generation, we focus on generating a rhythmically coherent bass line based on an input melody. The input melody and output bass line are both represented as sequences of discrete tokens: pitch values with a token reserved specifically for padding, and is ultimately batched via PyTorch's DataLoader

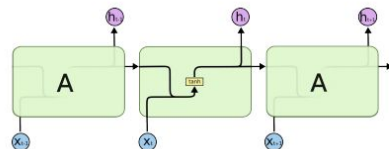


Our LSTM utilizes the Encoder-Decoder concept for the melody to bass generation. We feed a melody one note at a time so that the LSTM remembers what happens. At every step, the LSTM's memory is sent through a linear layer, picking bass notes that are most likely to occur in that moment. So, the LSTM encodes the melody, and its linear later decodes it into bass at each time stamp

Vanilla RNNs vs LSTMs

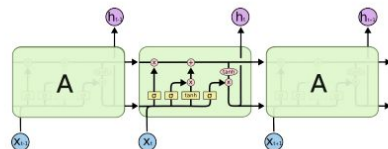
Vanilla RNNs

- Simplest form of RNNs
- Limited long-term memory



LSTMs

- Improved memory module
- Better long-term memory



Because LSTM's greater ability to learn and recall information on longer sequences, it is a more effective option for producing consistent bass lines. Normal RNNs forget too quickly, so it would be outputting less uniform melodies

Part 2. Modeling



Discussion & Code [Optimization & Parameter Tuning]

As we utilized the Melody to Bass LSTM, there were key parameters we changed for better optimization, training time, and overall computational resourcefulness.

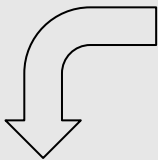
1. **seq_len:** Each training was done on a fixed-length chunk of 256 16th-note steps, as longer windows such as 512 would above optimal amounts of memory, while 128 began to shy away from the current harmonic context
2. **grid:** Limits the quantity of raw MIDI 16th-note steps with 120 ticks per step, ensuring the melody and bass line align correctly within the boundaries of the 16th-note steps
3. **embed_size:** Every tokened input is mapped to a 128-D (dimensional) embedding, which had better pitch relationships while also removing the concern of overfitting.
4. **hidden_size:** Because of the 256 steps, we store the context of the melodies in a 256-dimensional state, giving sufficient memory capacity
5. **num_layers:** We stacked 2 LSTM layers, finding it to be sufficient for our specific task without affecting the consistency
6. **num_epochs:** 20 passes through the training data, monitored using the training & validation loss, and found sufficient results
7. **batch_size:** 32 training samples used in each forward/backward pass, comfortably still fitting within GPU memory
8. **learning_rate (lr):** Controls how much the model's weight are adjusted w/ respect to the loss gradient, finding the 1e-3 to be the most reliable within our epochs

Part 2. Modeling



Discussion & Code [LSTM Comparison: Generated vs True Bass]

To evaluate our model, we create two sample melodies with differing bass lines: one generated by our model, and the other being the ground truth bass line from our built dataset. Note, both have identical melodies



```
mel = miditoolkit.MidiFile("samples/melody.mid")
pred = miditoolkit.MidiFile("samples/predB.mid")
```

```
merged = miditoolkit.MidiFile()
merged.ticks_per_beat = mel.ticks_per_beat
```

```
for inst in mel.instruments:
    merged.instruments.append(inst)
for inst in pred.instruments:
    merged.instruments.append(inst)
```

```
merged.dump("samples/predB.mid")
print("Wrote merged file: samples/melody_and_predB.mid")
```

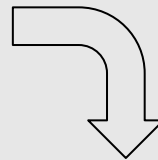


This is a code snippet showing the creation of our first audio comparison track. It reads the melody.mid (containing just the original melody) and predB.mid (containing the bass line our LSTM model predicted). These two are programmatically merged into a single two-track MIDI file, which we've titled melody_and_predB.mid. Listening to this file, we can hear how the model's generated bass line accompanies the melody

```
idx = random.randint(0, len(val_ds) - 1)
mel_chunk, trueB = val_ds[idx]
predB = generate_bass_from_melody(model, mel_chunk, device)

os.makedirs("./samples", exist_ok=True)
tokens_to_midi(trueB.tolist(), "./samples/trueB.mid")
tokens_to_midi(predB, "./samples/predB.mid")
tokens_to_midi(mel_chunk.tolist(), "./samples/melody.mid")
```

To set up our comparison, we first randomly select a melody (mel_chunk) and its source ground truth bass line (trueB_tokens) from our test set. We then apply our trained MelodyToBassLSTM model to generate a predicted bass line (predB_tokens) for this specific mel_chunk. Finally, all three distinctive pieces, the original melody, the true bass token sequence, and the predicted bass token sequence, are rendered and saved as individual MIDI files (melody.mid, trueB.mid, and predB.mid, respectively). These are the components for the merged tracks.



```
mel = miditoolkit.MidiFile("samples/melody.mid")
trueB = miditoolkit.MidiFile("samples/trueB.mid")
```

```
merged = miditoolkit.MidiFile()
merged.ticks_per_beat = mel.ticks_per_beat
```

```
for inst in mel.instruments:
    merged.instruments.append(inst)

for inst in trueB.instruments:
    merged.instruments.append(inst)
```

```
merged.dump("samples/trueB.mid")
print("Wrote merged file: samples/melody_and_trueB.mid")
```



As a point of reference, this code block creates the ground truth comparison track. It opens the same melody.mid and the trueB.mid (which contains the original bass line from the dataset that corresponds to the selected melody). It merges these into a third two-track MIDI file, melody_and_trueB.mid. This lets us hear the melody with its original, human-written (or dataset-provided) bass line, giving us a basis for comparison with our model's output

Evaluation



Why Our Method vs Simple Baselines

Random/Uniform Baseline:

- Choose a bass token at random each step (1/129 chance per note)
- Yields approx. 0.8% accuracy – this is our “floor”

Melody-12 Semitone Baseline:

- Predict the melody note minus one octave (clamped to [0,127])
- Achieves only approx. 8-12% accuracy, because sometimes bass really is an octave below but far from always

Always-Root Baseline:

- Predict a fixed pitch (MIDI 60) whenever a note is required
- Yields approx. 6% accuracy – better than random but still trivial

Our LSTM Model:

- Learns melodic -> bass patterns from data
- Achieves approx 28% token accuracy, compared to 0.8% / 8-12% / 6% for baselines demonstrating it has captured real harmonic structure

What Makes a “Good” Bass Line?

Harmonic Coherence:

- Should imply the correct chord under the melody
- High probability assigned by model to ground – truth bass sequences reflect this

Rhythmic Alignment:

- Notes placed on strong beats and synced with melodic accents (not randomly scattered)
- Low cross-entropy does not guarantee perfect rhythm, so we also listen for steady groove

Smooth Voice Leading & Pitch Transitions:

- Model’s generated sequences exhibit small intervals and stepwise motion in the bass

Objective vs Subjective:

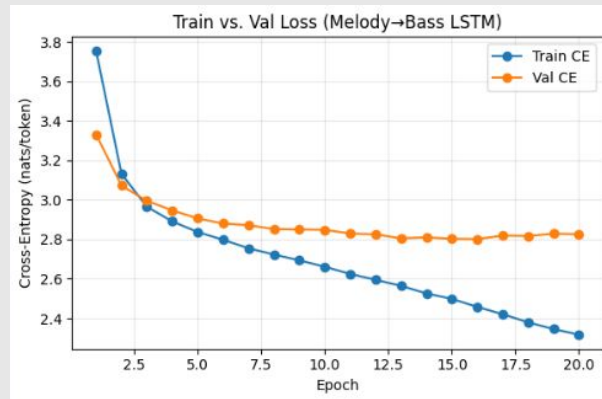
- Token Accuracy / Perplexity -> how often we match exactly the pianist’s bass tokens
- But an “incorrect” token (ex/ chord fifth instead of root) can still sound musically acceptable
- Hence we pair quantitative scores with **qualitative listening** of “melody + predicted bass” vs “melody + true bass”

Evaluation



LSTM Training & Validation Performance

- Rapid learning in early epochs: In both the training and validation cross-entropy (CE) losses drop quickly during the first few epochs. Training CE drops from ~3.75 to ~2.6 by epoch 8. Validation CE drops from ~3.15 to ~2.85 by epoch 8.
- Elbow at epoch 8: After epoch 8, validation loss plateaus at ~2.8, while training loss continues to improve.
- Overfitting begins: The widening gap between training and validation loss suggest reduced generalization
- Checkpoint rationale: We save the model at epoch 8 – the lowest validation loss – for best generalization



What Makes Our Music “Good”?

- Our Code computes LSTM token accuracy on the validation set and compares to each baseline’s accuracy
- We reinforce numeric results (~28% vs 0.8-12%) with audio demos so listeners can judge true musical quality
- A significantly lower cross-entropy and higher token accuracy indicate our LSTM learned genuine melodic -> harmonic relationships between trivial rules

Part 4: Discussion of Related Work



How has this data set been used before?

In the past, the MAESTRO dataset had been used for:

- Automatic music transcription: Training models, such as the *Onsets and Frames* piano transcription model, to convert audio to MIDI with high accuracy.
- Symbolic music modeling: Training generative models, such as a *Music Transformer* model, on MIDI token sequences for note prediction and sequence generation.
- Audio synthesis: Using MIDI as input to models, such as a *WaveNet* model, to generate high-quality waveform generation.

How has prior work approached the same (or similar) tasks?

The Wave2Midi2Wave pipeline is a key example of prior work that approached similar tasks. The Wave2Midi2Wave pipeline uses the MAESTRO dataset in order to train a collection of models designed to transcribe, compose, and synthesize audio waveforms while maintaining coherent musical structure across timescales ranging from approximately 0.1 milliseconds to 100 seconds.

- Wave2Midi: Transcribes piano audio into MIDI by training the *Onsets and Frames* piano transcription model on the MAESTRO dataset
- MiDI generation: Generates new piano performances by training a *Music Transformer* model on the transcriptions of the piano music it received from the Wave2Midi
- Midi2Wave: Synthesizes realistic piano audio from MIDI inputs through a trained *WaveNet* model

Part 4: Discussion of Related Work

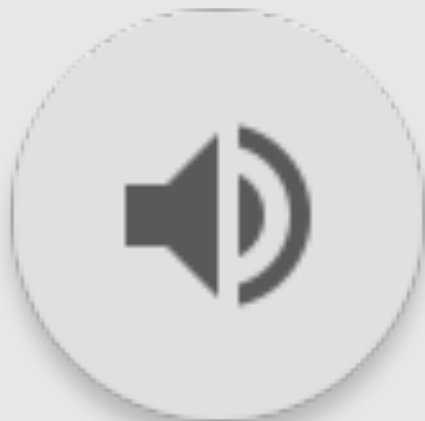


How does your work match or differ from what has been reported in related work?

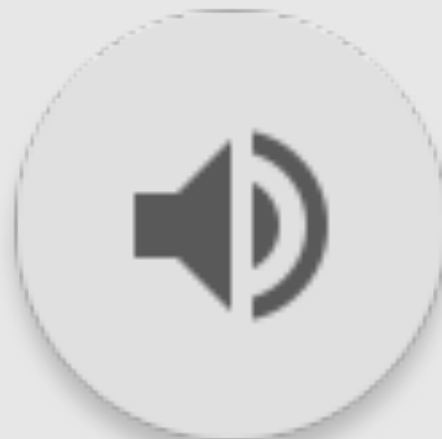
Our work in our symbolic, conditioned generation model is similar to the approach used in the Wave2Midi2Wave pipeline. In the Wave2Midi2Wave pipeline, one of the core components is a *Music Transformer* model trained on transcriptions of piano performances from the MAESTRO dataset. This model learns the structure and patterns in symbolic music data to generate new coherent and realistic piano performances. Similar to Wave2Midi2Wave, our symbolic, conditioned generation model also used a Music Transformer to generate new piano performances from symbolic MIDI data. Like that work, we also use a Transformer-based architecture trained on tokenized MIDI files to model musical structure. This allowed us to generate coherent, style-consistent music generation which aligned closely with the Wave2Midi2Wave pipeline.

In contrast, our symbolic, unconditioned model generates MIDI sequences by starting from a random noise or start token, without any input context. This resembles earlier symbolic music generation methods, such as Markov chains or LSTM models, that sample sequences from learned musical distributions. Unlike conditioned models, unconditioned models offer more open-ended creativity but can result in less musically coherent output.

Our Generated Outputs



Symbolic Unconditional



Symbolic Conditional

THANK
YOU!

